

UNIVERSITÀ DEGLI STUDI DI ROMA “TOR VERGATA”

Macroarea di Ingegneria

CORSO DI LAUREA IN INGEGNERIA INFORMATICA

INGEGNERIA DEL SOFTWARE E PROGETTAZIONE WEB (ISPW)

CIBOAMICO

*PIATTAFORMA PER LA GESTIONE DEL CIBO IN CASA, LA RIDUZIONE
DEGLI SPRECHI E L'OTTIMIZZAZIONE DELLA SPESA*

DOCENTI:

PROF. DAVIDE FALESSI

PROF. GUGLIELMO DE ANGELIS

AUTORE:

MICHELE DAMIANO

MATRICOLA: 0324516

MICHELE.DAMIANO@STUDENTS.UNIROMA2.EU

ANNO ACCADEMICO 2025/2026

DATA DI CONSEGNA: 11 AGOSTO 2026

Indice

1	Specifiche dei Requisiti Software	3
1.1	Introduzione	3
1.1.1	Scopo del documento	3
1.1.2	Panoramica del sistema	3
1.1.3	Requisiti hardware e software	4
1.1.4	Sistemi correlati	4
1.2	User Stories	5
1.3	Requisiti Funzionali	5
1.3.1	Glossario dei Termini di Dominio	6
1.4	Use Cases	6
1.4.1	Diagramma dei casi d'uso	6
1.4.2	Internal Steps	6
2	Storyboards	8
2.1	Autenticazione e Area Personale	8
2.1.1	Autenticati	8
2.1.2	Home (Cliente)	8
2.2	Acquisti e Marketplace (UC-04 Ordina un Prodotto)	9
2.2.1	Marketplace (Catalogo Prodotti)	9
2.2.2	Ordina un Prodotto	10
2.2.3	Pagamento (passo 6)	11
3	Design	13
3.1	Class Diagram	13
3.1.1	VOPC (Analysis)	13
3.1.2	Design Patterns	13
3.2	Activity Diagram	17
3.3	Sequence Diagram	18
3.4	State Diagram	18
4	Testing	19

5	Exceptions	21
5.1	Wrapping	21
5.2	Payment	21
5.3	Messaggi	22
6	Persistenza	23
6.1	Layer di persistenza DB (MySQL/JDBC)	23
6.2	Layer di persistenza FS (File System)	23
6.3	Layer di persistenza DEMO (In-Memory)	24
7	Video e Link	25
7.1	Repository GitHub	25
7.2	SonarCloud	25
7.3	Video dimostrativo	25

Capitolo 1

Specifiche dei Requisiti Software

1.1 Introduzione

1.1.1 Scopo del documento

Questo documento definisce i requisiti software di **CiboAmico**, il progetto finale del corso ISPW (Università di Roma “Tor Vergata”, A.A. 2025/2026, Prof. Davide Falessi, Prof. Guglielmo De Angelis).

Lo scopo è fissare cosa il sistema deve fare in modo chiaro e verificabile, lasciando campo alla progettazione e ai test. Il documento serve sia a chi sviluppa, per tenere fede all’architettura Boundary-Control-Entity (BCE), sia a chi valuta il prodotto, per confrontare i flussi funzionali con le richieste iniziali.

1.1.2 Panoramica del sistema

CiboAmico è un’applicazione desktop JavaFX che aiuta a gestire il cibo in casa, evitare sprechi e ottimizzare la spesa, collegando l’utente ai commercianti di prossimità. Il sistema segue il pattern Boundary-Control-Entity (BCE) e coinvolge quattro attori: Cliente, Venditore, Nutrizionista e Amministratore. Due sistemi esterni sono raggiunti tramite Adapter: OpenFoodFacts (lettura dei dati nutrizionali dal codice a barre) e JakartaMail (invio di notifiche). La persistenza è intercambiabile a runtime tra una modalità *Demo* (in-memory) e una *Full* (MySQL via JDBC).

Il sistema ha tre aree funzionali:

- **Dispensa:** il Cliente traccia i prodotti in casa, vede le scadenze e viene avvisato prima che il cibo si deteriori;
- **Ricette:** il sistema propone ricette compatibili con gli ingredienti disponibili, sostituendo quelli mancanti;

- **Marketplace:** i Venditori pubblicano i prodotti, i Clienti fanno ordini diretti (un compratore, un venditore) e l'Amministratore approva venditori e ricette.

1.1.3 Requisiti hardware e software

L'applicazione è cross-platform grazie alla JVM.

Requisiti software:

- Java 17+
- JavaFX 17+ (per l'interfaccia desktop)
- Maven 3.9+ (build tool)
- MySQL Server 8.0+ (modalità *Full*, via JDBC)
- Draw.io (per la creazione dei diagrammi UML)
- IDE: IntelliJ IDEA (con EcEmma per la coverage) o Eclipse

Requisiti hardware:

- CPU: Dual-Core 2.0 GHz a 64-bit
- RAM: 2 GB (4 GB consigliati in modalità *Full* con server MySQL locale)
- Spazio su disco: 1 GB
- Schermo: 1024×768
- Sistema operativo: Windows, macOS, Linux (compatibile con Java 17+)

1.1.4 Sistemi correlati

Di seguito due sistemi esistenti che coprono solo in parte quello che fa CiboAmico.

System 1: Too Good To Go

Pro:

- Rete capillare di partner con offerte geolocalizzate in tempo reale.
- Riduce attivamente lo spreco commerciale offrendo cibo a prezzi ridotti.

Contro:

- Non traccia la dispensa di casa: dopo l'acquisto non aiuta a gestire le scadenze.
- L'acquisto è una *Magic Box* a sorpresa: non si scelgono i singoli prodotti né si pianificano ricette o diete.

System 2: SuperCook

Pro:

- Matching potente: trova subito molte ricette a partire dagli ingredienti inseriti.
- Ampio database con filtri per tipo di piatto e intolleranze.

Contro:

- Non ha un canale di acquisto integrato: per gli ingredienti mancanti bisogna provvedere da soli.
- Non sostituisce automaticamente gli ingredienti, escludendo ricette realizzabili con piccole variazioni.

CiboAmico punta a combinare tracciamento della dispensa, generazione di ricette con sostituzioni e acquisto diretto dai produttori locali in un'unica piattaforma, offrendo funzionalità dedicate a clienti, venditori, nutrizionisti e amministratori, mantenendo un'architettura modulare ed estensibile.

1.2 User Stories

1. **US-1:** Come Cliente, voglio ricevere notifiche preventive sulle date di scadenza dei prodotti presenti nella mia dispensa, in modo che possa consumare gli alimenti in tempo ed evitare lo spreco di cibo.
2. **US-2:** Come Cliente, voglio visualizzare le ricette compatibili con gli ingredienti disponibili in dispensa, in modo che possa cucinare senza dover effettuare nuovi acquisti.
3. **US-3:** Come Cliente, voglio ordinare un prodotto direttamente da un venditore locale, in modo che possa ricevere cibo fresco proveniente dal mio territorio.

1.3 Requisiti Funzionali

1. **RF-1:** Il sistema deve confrontare quotidianamente le date di scadenza degli alimenti nella dispensa con la data corrente, inviando un avviso al cliente per ogni prodotto la cui scadenza sia entro tre giorni.
2. **RF-2:** Il sistema deve suggerire al cliente un elenco di ricette in cui gli ingredienti richiesti siano un sottoinsieme degli ingredienti non scaduti presenti nella dispensa.
3. **RF-3:** Il sistema deve registrare una nuova transazione di acquisto associando il cliente al venditore e decrementando la scorta del prodotto in misura pari alla quantità ordinata.

1.3.1 Glossario dei Termini di Dominio

Dispensa : rappresentazione digitale degli alimenti presenti nell'inventario domestico dell'utente, con quantità e date di scadenza.

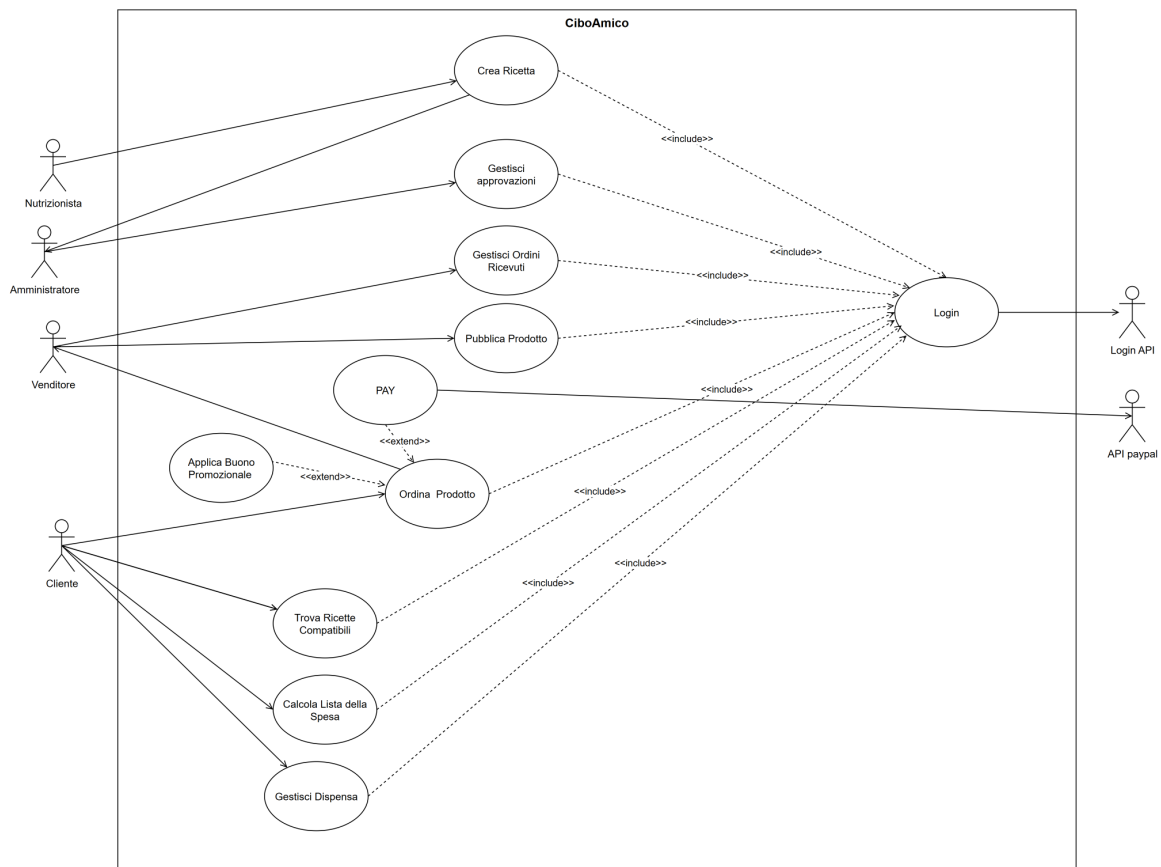
Ricetta compatibile : ricetta i cui ingredienti richiesti sono presenti, oppure sostituibili in modo equivalente, nella dispensa dell'utente.

Ordine diretto : transazione che associa un singolo cliente acquirente a un unico venditore, senza passare da un carrello condiviso o intermediari.

Disponibilità del prodotto : quantità di un prodotto attualmente vendibile, che non può mai essere negativa e viene aggiornata dopo ogni acquisto.

1.4 Use Cases

1.4.1 Diagramma dei casi d'uso



1.4.2 Internal Steps

Ordina un Prodotto

Main Flow:

1. The customer accesses the marketplace via Login.
2. The system displays the available products with price and remaining stock.
3. The customer selects a product from the catalog and starts the checkout.
4. The system loads the selected product into the current order and computes the total amount.
5. The customer confirms the purchase.
6. The system requests the authorization for the debit of the computed amount from the Payment Service.
7. The system registers the order in the “CREATED” state, decreases the remaining stock of the product and notifies the vendor.

Preconditions: the customer is authenticated; at least one product is available in the catalog.

Alternative Flow (Extensions):

- **3a. Product No Longer Available:** the selected product is not present in the catalog anymore; the system displays an error message.
- **4a. Promotional Voucher Request:** the customer requests to apply a promotional voucher to the current order; the system validates the temporal validity of the voucher and its association with the vendor of the selected product, recalculates the total amount applying the discount and resumes from step 5.
- **6a. Payment Rejected:** the Payment Service returns a negative outcome; the system cancels the current order and displays a failure message, returning the flow to step 4.

Capitolo 2

Storyboards

2.1 Autenticazione e Area Personale

2.1.1 Autenticati

La schermata di accesso permette all'utente di autenticarsi inserendo le proprie credenziali (email e password).

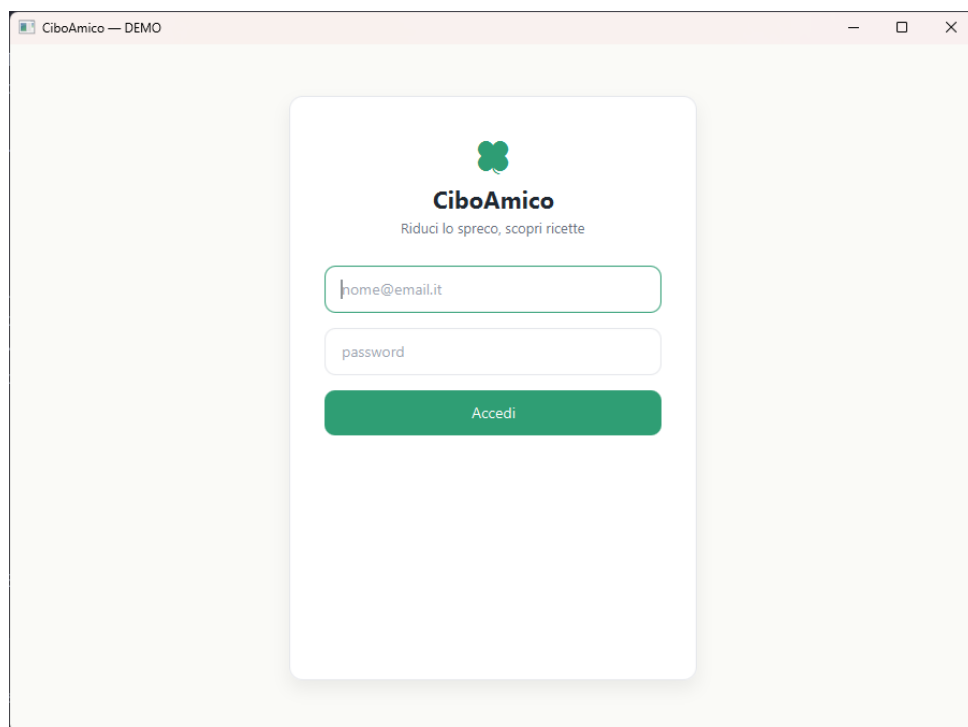


Figura 2.1: Schermata di autenticazione al sistema.

2.1.2 Home (Cliente)

Completata l'autenticazione, il Cliente raggiunge la dashboard principale (Home). Da qui può raggiungere le funzionalità dell'applicazione: Ricette, Inventario, Lista Spesa e

Marketplace (UC-04 Ordina un Prodotto).

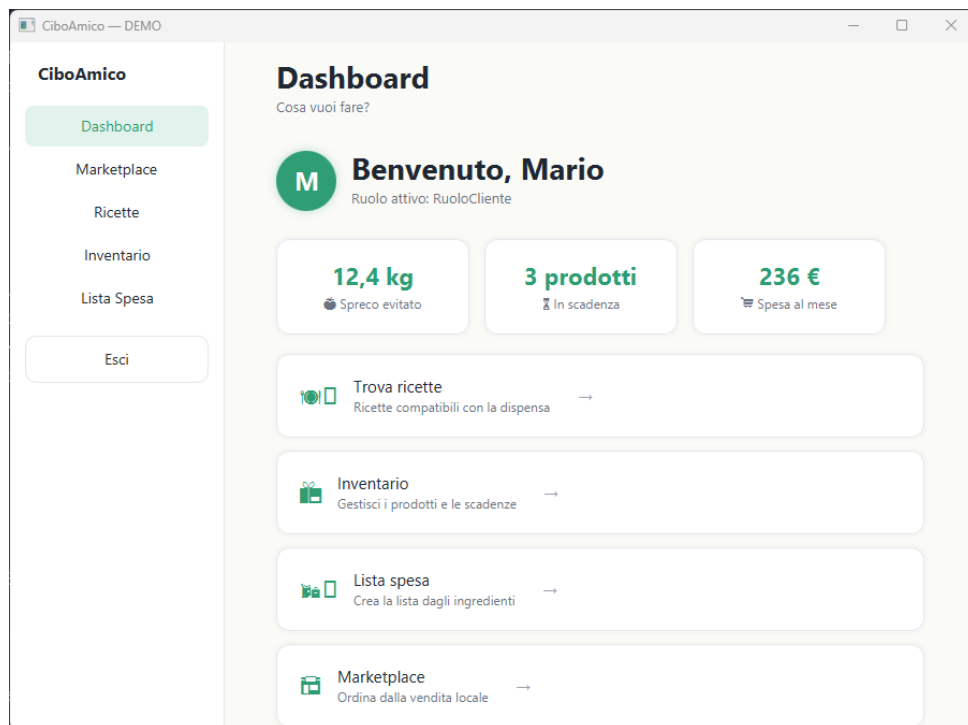


Figura 2.2: Dashboard principale: accesso alle funzionalità dell'applicazione.

2.2 Acquisti e Marketplace (UC-04 Ordina un Prodotto)

2.2.1 Marketplace (Catalogo Prodotti)

L'area dedicata all'acquisto diretto mostra il catalogo dei prodotti locali dei venditori. Il pannello laterale consente di selezionare un prodotto da ordinare e, opzionalmente, un buono promozionale.

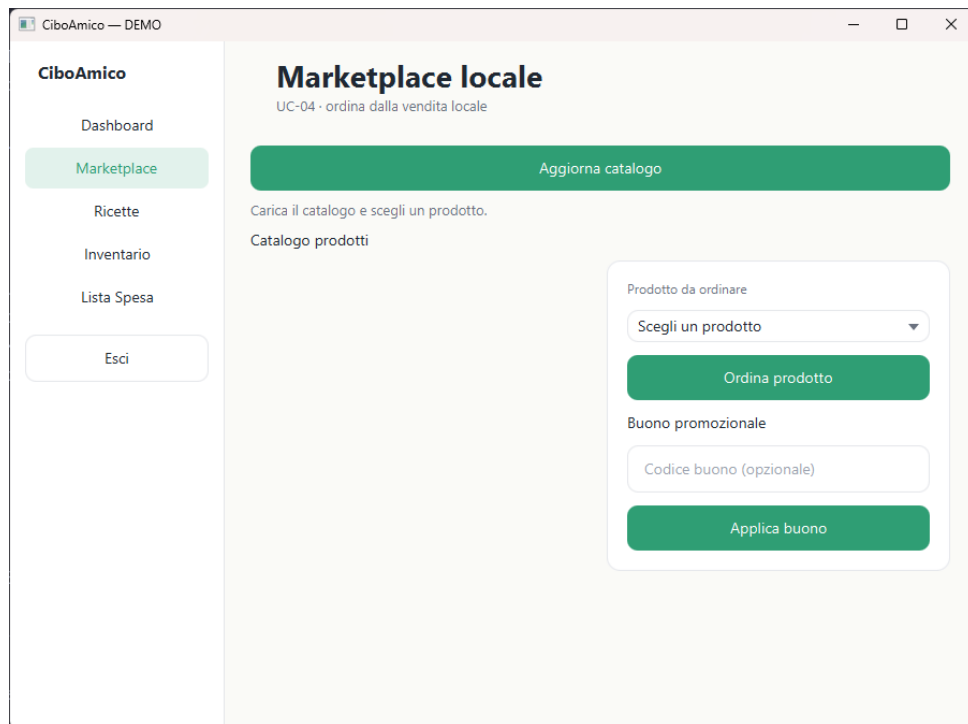


Figura 2.3: Marketplace: catalogo dei prodotti locali e pannello di acquisto.

2.2.2 Ordina un Prodotto

Selezionato un prodotto dal catalogo e caricata la lista (pulsante *Aggiorna catalogo*), il Cliente sceglie il prodotto nel pannello *Prodotto da ordinare*. Se inserisce un codice buono e preme *Applica buono*, il sistema valida il buono e ricalcola il totale applicando lo sconto (estensione *Ordina un Prodotto*); con *Ordina prodotto* procede.

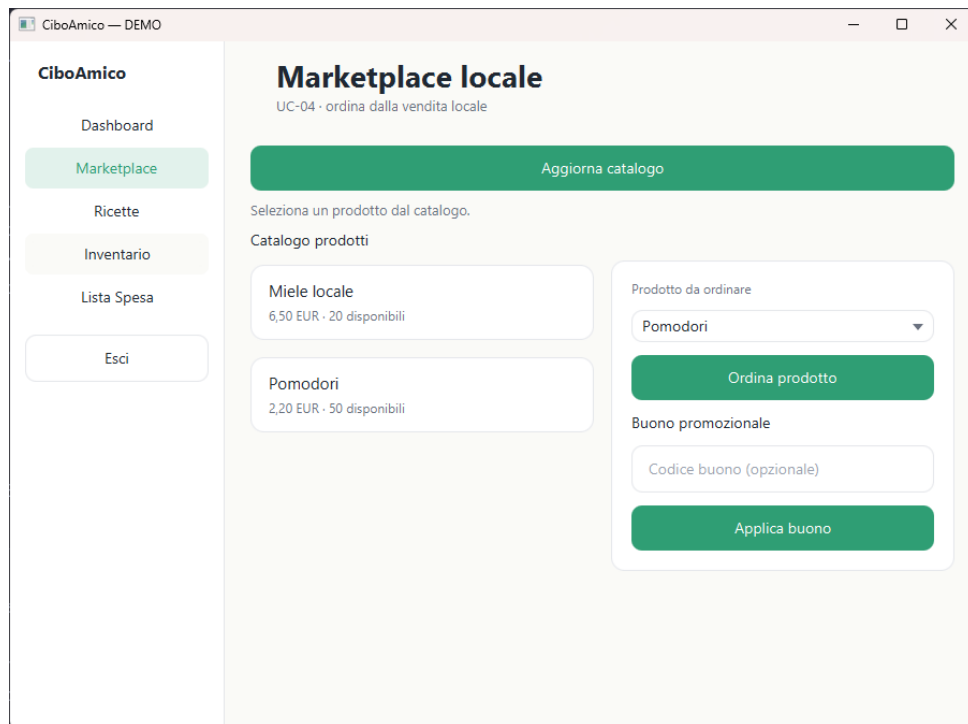


Figura 2.4: Schermata di acquisto con prodotto selezionato e campo buono promozionale.

2.2.3 Pagamento (passo 6)

Confermato l'ordine, l'interfaccia passa alla schermata di pagamento: l'utente inserisce i dati della carta (numero, intestatario, scadenza e CVV) e autorizza l'addebito; l'esito è mostrato inline nella stessa vista, come richiesto dall'estensione **6a** (pagamento rifiutato) del caso d'uso.

The screenshot shows a web application window titled "CiboAmico — DEMO". On the left is a sidebar menu with the following items: "Dashboard", "Marketplace" (highlighted in green), "Ricette", "Inventario", "Lista Spesa", and "Esci". The main content area is titled "Pagamento" and includes the subtitle "UC-04 · autorizzazione all'addebito". Below this, a summary line reads "Riepilogo: Pomodori — 2,20 EUR". The form contains several input fields: "Numero carta", "Intestatario", "Scadenza (MM/AA)", and "CVV". At the bottom of the form are two large green buttons labeled "Paga" and "Annulla".

Figura 2.5: Schermata di pagamento (dati carta e autorizzazione all'addebito).

Capitolo 3

Design

3.1 Class Diagram

3.1.1 VOPC (Analysis)

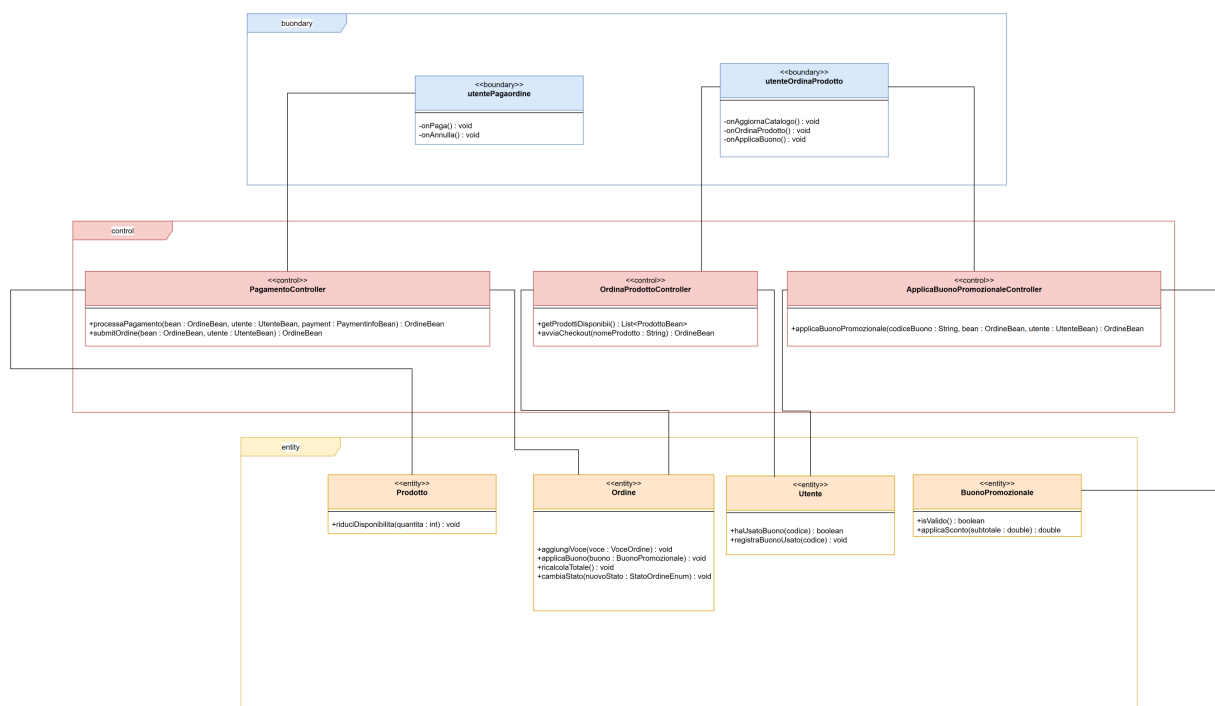


Figura 3.1: VOPC di UC-04: Boundary (View), controller grafico, Facade, controller applicativi ed Entity.

3.1.2 Design Patterns

Il design-level introduce le interfacce, i tipi Java, le eccezioni e i pattern GoF per gestire estensibilità e persistenza dinamica (NFR-01). Per ogni pattern sono mostrati in ordine lo schema e la breve descrizione dell'applicazione.

Abstract Factory Pattern – Persistence Layer

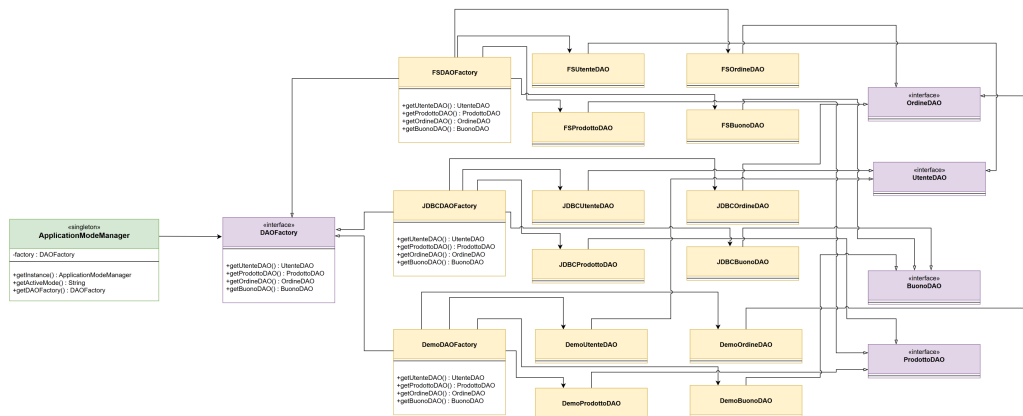


Figura 3.2: Abstract Factory della persistenza: interfacce DAO, concrete DAO e factory.

Il livello di persistenza è gestito tramite il pattern **Abstract Factory** al fine di garantire flessibilità di configurazione dell'ambiente di esecuzione. L'interfaccia `DAOFactory` definisce il contratto per la creazione dei vari Data Access Object, come `getOrdineDAO()` e `getProdottoDAO()`. In base alla proprietà `persistence_type`, le factory concrete (`JDBCDAOFactory`, `FSDAOFactory`, `DemoDAOFactory`) istanziano l'opportuna implementazione dello storage. Ciò consente al sistema di passare senza problemi tra una persistenza basata su database, file-system o memoria in-memory senza richiedere alcuna modifica al codice client.

Observer Pattern – Notification System

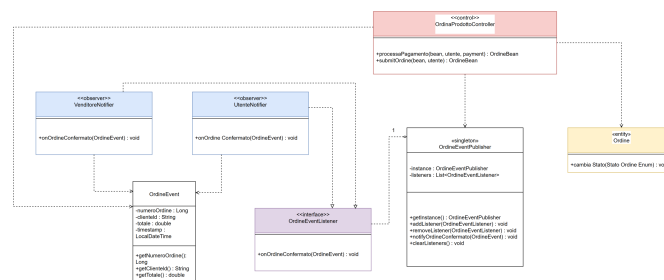


Figura 3.3: Observer della conferma dell'ordine: publisher singleton e DTO.

Il pattern **Observer** è stato implementato con lo scopo di disaccoppiare l'elaborazione dell'ordine dalla logica di gestione delle notifiche. La classe `OrderEventPublisher` funge da *Subject* implementato tramite il pattern **Singleton**, gestendo una lista (`List`) di osservatori di tipo `OrderEventListener`. Al momento del corretto invio e della conferma dell'ordine, il publisher attiva il metodo `onOrderConfermato()` su tutti i listener registrati (compratore e venditore), passando il DTO `OrderEvent` in sola lettura, mai l'entità di

dominio. Questa architettura garantisce un accoppiamento debole tra la logica di business principale e i relativi effetti collaterali, rispettando rigorosamente l'*Open/Closed Principle*.

La notifica è emessa dal sottosistema applicativo (il control che conferma l'ordine) tramite il publisher, e non dall'entity **Ordine**: cos` l'entità di dominio resta *svincolata* dal layer di presentazione e dal meccanismo di notifica (separazione tra interfaccia grafica e struttura dati indicata in teoria). La variante è quella *publish-subscribe*/event-driven dell'Observer: l'accoppiamento tra **OrdineEventPublisher** (Subject) e gli observer è astratto (solo **OrdineEventListener**), la notifica è un broadcast a tutti i listener registrati e il dato scambiato è il DTO **OrdineEvent** in sola lettura, mai l'entità.

Singleton Pattern – Sessions and Mode

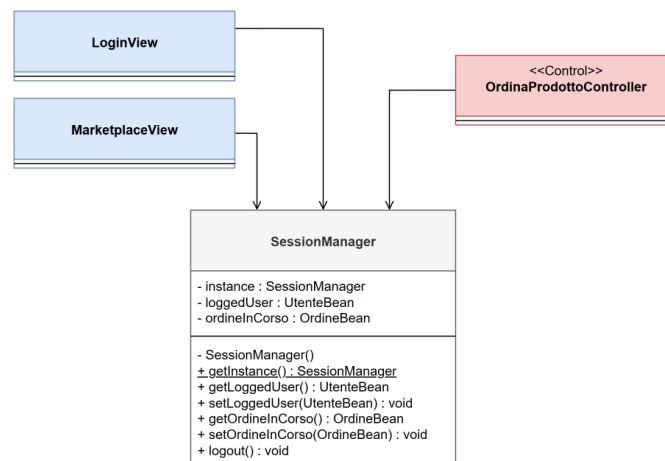


Figura 3.4: Singleton della sessione e della modalità.

SessionManager (sessione utente) e **ApplicationModeManager** (modalità applicativa) adottano un singleton classico con *lazy initialization* (`if (instance == null) instance = new X()`) e sincronizzazione su `getInstance()`, in linea coi progetti di riferimento: nessun costo percepibile in un client desktop, nessuna soppressione di warning per SonarCloud.

Facade Pattern – Checkout UC-04

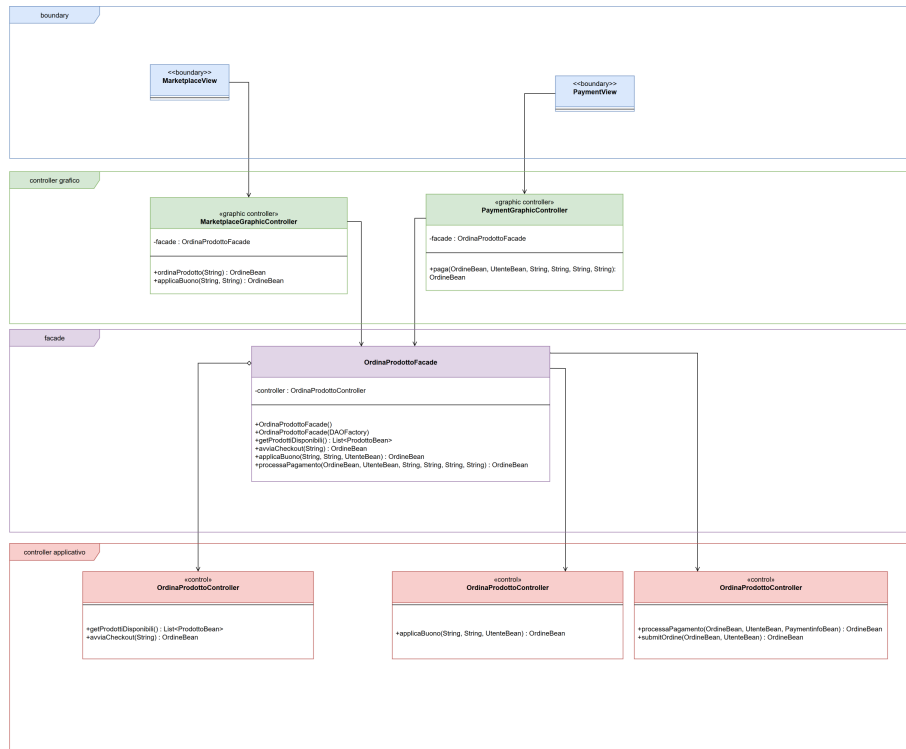


Figura 3.5: Facade del checkout di UC-04.

`OrdinaProdottoFacade` espone al controller grafico un'unica interfaccia per il checkout (`getProdottiDisponibili`, `avviaCheckout`, `applicaBuono`, `processaPagamento`) e nasconde il sottosistema di business, formato dal controller base e dai controller delle estensioni (buono 4a e pagamento 6/6a). Il client (boundary o controller grafico) non conosce l'ordine di chiamata ai metodi dei controller applicativi.

3.2 Activity Diagram

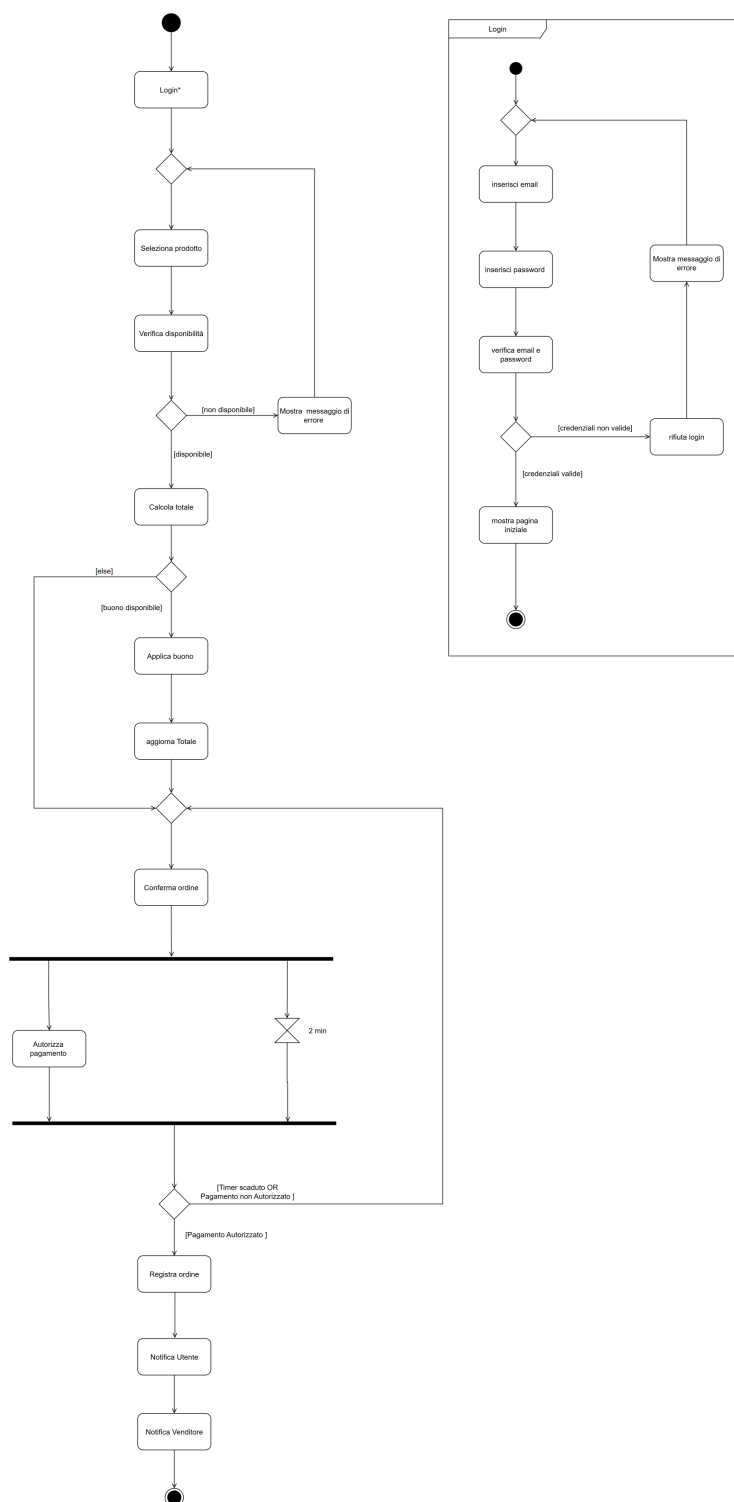


Figura 3.6: Activity diagram di UC-04: verifica disponibilità, applicazione del buono, autorizzazione del pagamento e notifica sequenziale a compratore e venditore.

3.3 Sequence Diagram

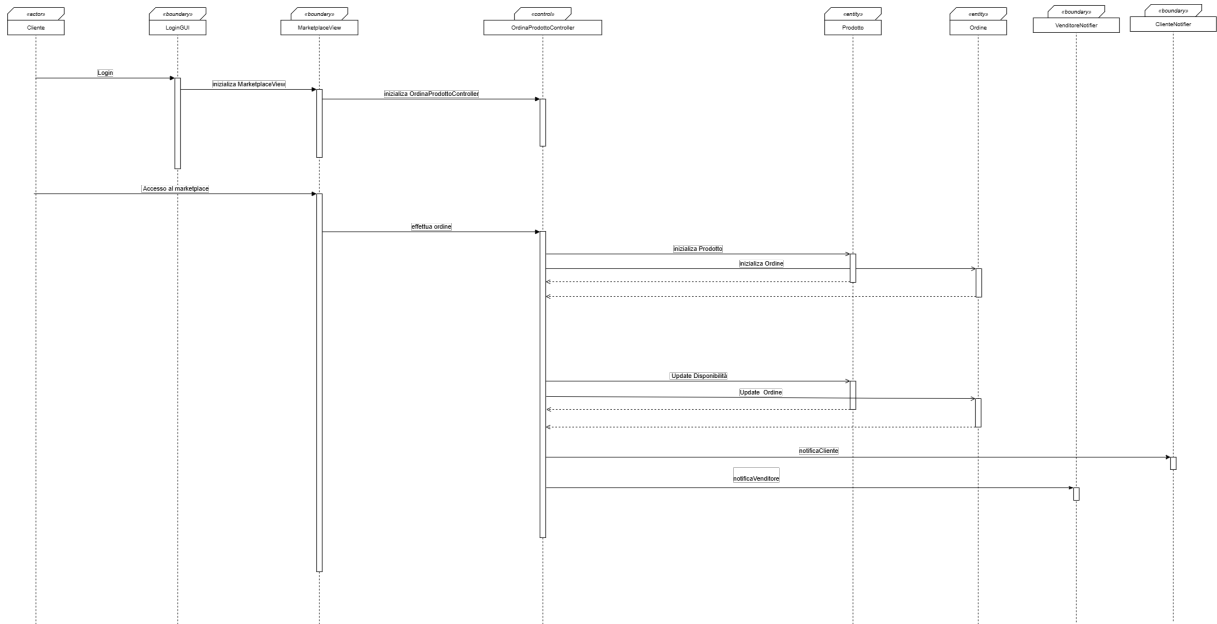


Figura 3.7: Sequence diagram di UC-04.

3.4 State Diagram

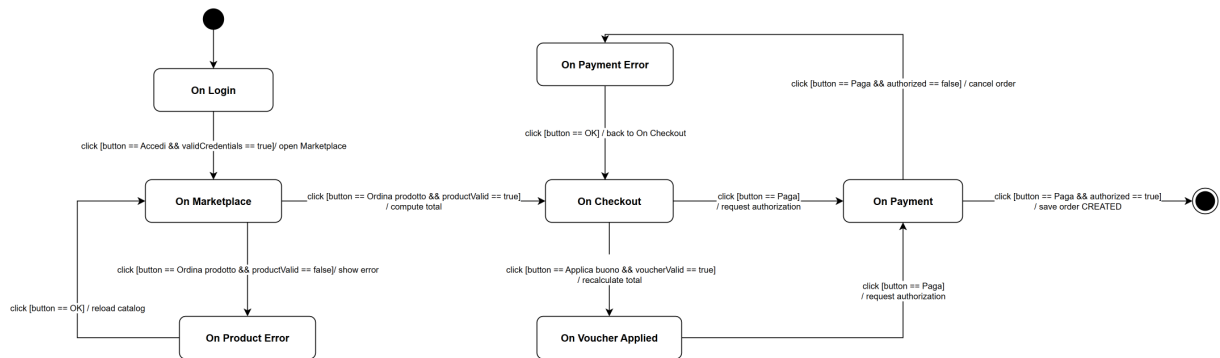


Figura 3.8: State diagram di UC-04 Ordina un Prodotto: navigazione e flussi alternativi 3a, 4a e 6a.

Capitolo 4

Testing

La verifica del sistema è condotta con una suite di unit test **JUnit 5** eseguita da Maven e con copertura misurata da JaCoCo (quality gate su copertura di riga). Di seguito i comportamenti verificati, descritti per funzionalità.

Accesso all'area riservata. Abbiamo testato il controller dell'area riservata verificando che un utente autenticato recuperi correttamente i dati del proprio checkout e della propria sessione. I test garantiscono che le strutture restituite non siano mai `null` e che l'accesso ai dati riservati sia gestito correttamente per gli utenti autenticati.

Creazione e sottomissione dell'ordine. Abbiamo testato il flusso di ordinazione verificando che un prodotto venga selezionato e che l'ordine venga creato con il venditore corretto. I test garantiscono che un ordine con utente o prodotto nullo venga respinto e che la disponibilità di magazzino sia ridotta all'acquisto senza scorte negative.

Scontistica del buono (Strategy). Abbiamo testato la logica di scontistica verificando che un buono valido venga applicato al totale dell'ordine. I test garantiscono che codici vuoti, scaduti o non validi vengano respinti senza eccezioni non gestite e che il calcolo dello sconto non produca mai importi incoerenti.

Pagamento e autorizzazione. Abbiamo testato il sottosistema di pagamento verificando sia il ramo autorizzato sia quello rifiutato. I test garantiscono che un pagamento rifiutato sollevi l'eccezione applicativa corretta lasciando invariato l'ordine, e che dati di carta non validi (CVV, importo non positivo, ordine assente) vengano bloccati senza `NullPointerException`.

Autenticazione. Abbiamo testato il meccanismo di autenticazione verificando sia credenziali valide sia non valide. I test garantiscono che un login corretto popoli la sessione e

che email o password errate vengano rifiutate, con la `logout` che pulisce correttamente la sessione.

Ciclo di vita dell'ordine (macchina a stati BR-04). Abbiamo testato la macchina a stati dell'ordine verificando la sequenza `CREATED` $\rightarrow$ `CONFIRMED` $\rightarrow$ `IN_DELIVERY` $\rightarrow$ `DELIVERED`. I test garantiscono che una transizione illegale venga bloccata dall'eccezione di stato invalido.

Notifica Observer. Abbiamo testato il pattern Observer verificando che il publisher notifichi gli observer registrati quando un ordine viene confermato. I test garantiscono che l'evento (`OrdineEvent`) venga consegnato agli observer tramite il DTO in sola lettura, mai l'entità di dominio.

Persistenza (Demo e File System). Abbiamo testato i livelli di persistenza verificando il ciclo crea-leggi-aggiorna di utenti, prodotti e ordini. I test garantiscono che la persistenza *Demo* (in-memory) e *File System* mantengano i dati in modo coerente, incluso il recupero dei prodotti per nome.

Capitolo 5

Exceptions

La logica di errore è una tassonomia (`exception/`) tutta **checked**, che forza la gestione esplicita (`throws / try/catch`).

Eccezione	Ruolo
<code>CiboAmicoException</code> (abstract)	Base checked, campi comuni
<code>BusinessValidationException</code>	Regola di business violata
<code>AutenticazioneException</code>	Credenziali errate
<code>InvalidStateTransitionException</code>	Stato illegale (BR-04)
<code>DAOException</code>	Persistenza (file / DB)

La base `CiboAmicoException` estende `java.lang.Exception` (**checked**) ed espone quattro campi: `userMessage` (mostrato all'utente), `technicalMessage` (per i log), `errorCode` e `timestamp`. Le view leggono solo `getUserMessage()`.

5.1 Wrapping

I DAO incapsulano l'errore di basso livello (`SQLException`, `IOException`) in `DAOException` con (`message`, `cause`): `getCause()` preserva la causa originale, che non si propaga mai alle boundary.

5.2 Payment

`PaymentRejectedException` (`pattern.payment`) è checked: transazione non autorizzata, gestita dall'estensione 6a di UC-04.

5.3 Messaggi

Testi centralizzati in `ExceptionMessagesEnum` (log) e `UserErrorMessagesEnum` (utente), senza stringhe duplicate nei punti di lancio.

Capitolo 6

Persistenza

La persistenza è intercambiabile a runtime (NFR-01): tre backend mutuamente esclusivi sono selezionati tramite il pattern **Abstract Factory** (`DAOFactory` + una concrete factory per modalità).

6.1 Layer di persistenza DB (MySQL/JDBC)

Il livello di persistenza DB utilizza **MySQL** come DBMS relazionale. La gestione dei dati è affidata a classi DAO specifiche (es. `JDBCUserDAO`, `JDBCOrdineDAO`), che eseguono query SQL per le operazioni CRUD sulle entità.

Entità principali:

- **Utenti:** registrati, con i ruoli (Cliente e Venditore);
- **Ordini:** record transazionali con stato, totale e voci;
- **Prodotti:** articoli del marketplace (catalogo e inventario);
- **Buoni:** codici sconto (percentuale o importo fisso).

Le transazioni sono atomiche (`ConnectionManager` usa `setAutoCommit(false)` con `commit/rollback` per la riduzione scorte).

Configurazione: il sistema è configurato via il file `config.properties` impostando `persistence_type = DB` e fornendo le credenziali MySQL necessarie.

6.2 Layer di persistenza FS (File System)

Il layer di persistenza FS memorizza i dati in file **JSON** nella cartella `data/`. Ogni entità è mappata su un file dedicato (es. `utenti.json`, `ordini.json`, `prodotti.json`, `inventario.json`).

Gestione: la classe `GsonConfig` fornisce un serializzatore JSON coerente (helper per `LocalDate/LocalDateTime`) usato da tutti i DAO FS per garantire la lettura/scrittura corretta dei record.

Configurazione: la modalità è attivata impostando `persistence_type = FS` nel file di configurazione.

6.3 Layer di persistenza DEMO (In-Memory)

Il layer di persistenza DEMO mantiene i dati esclusivamente nella memoria volatile (RAM), senza storage permanente. Le strutture sono popolate da valori predefiniti all'avvio e cancellate al termine dell'esecuzione.

Casi d'uso: è pensato per il testing unitario, lo sviluppo rapido e le dimostrazioni che non richiedono configurazione di database.

Configurazione: la modalità è attivata impostando `persistence_type = DEMO` (default).

Capitolo 7

Video e Link

7.1 Repository GitHub

<https://github.com/Damiano-o/ciboamico.git>

7.2 SonarCloud

<https://sonarcloud.io/organizations/ciboamico-ispw>

7.3 Video dimostrativo

Da inserire prima della consegna.